Introduction to Algorithms

Email: grad.saeed@gmail.com

Github: github.com/saeedgrad

Recurrences

- To analyze recursive divide-and-conquer algorithms, we'll need some mathematical tools.
- A recurrence is an equation that describes a function in terms of its value on other, typically smaller, arguments.
- The general form of a recurrence is an equation or inequality that describes a function over the integers or reals using the function itself.

Recurrences

- A recurrence contains two or more cases, depending on the argument. If a case involves the recursive invocation of the function on different (usually smaller) inputs, it is a recursive case.
- If a case does not involve a recursive invocation, it is a base case.
- The recurrence is well defined if there is at least one function that satisfies it, and ill defined otherwise.

Examples of recurrences

$$T(n) = \begin{cases} 1 & \text{if } n = 1 , \\ T(n-1) + 1 & \text{if } n > 1 . \end{cases}$$

$$T(n) = \begin{cases} 1 & \text{if } n = 1 , \\ 2T(n/2) + n & \text{if } n \geq 1 . \end{cases}$$

$$T(n) = \begin{cases} 0 & \text{if } n = 2 , \\ T(\sqrt{n}) + 1 & \text{if } n > 2 . \end{cases}$$

$$T(n) = \begin{cases} 1 & \text{if } n = 1 , \\ T(n/3) + T(2n/3) + n & \text{if } n > 1 . \end{cases}$$

Solving recurrences

1- Substitution method:

- 1. Guess the form of the solution using symbolic constants.
- 2. Use mathematical induction to show that the solution works, and find the constants.

Substitution method, example:

Recurrence:

$$T(n) = 2T(\lfloor n/2 \rfloor) + \Theta(n) .$$

1- Guess:

$$T(n) = O(n \lg n)$$

2- Induction:

In particular, therefore, if $n \geq 2n_0$, it holds for $\lfloor n/2 \rfloor$

$$T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor).$$

Substitution method, example:

$$
\begin{aligned}
T(n) &\le 2(c\,\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + \Theta(n) \\
&\le 2(c(n/2)\lg(n/2)) + \Theta(n) \\
&= cn\lg(n/2) + \Theta(n) \\
&= cn\lg n - cn\lg 2 + \Theta(n) \\
&= cn\lg n - cn + \Theta(n) \\
&\le cn\lg n\ ,
\end{aligned}
$$

Substitution method, merge sort:

Suppose that $n = 2^k$, for some $k \in \mathbb{Z}$.

$$
\begin{aligned}
T(n) &= 2T(n/2) + cn \\
&= 2T(2^{k-1}) + c2^k \\
&= 2\left(2T(2^{k-2}) + c2^{k-1}\right) + c2^k \\
&= 2^2 T(2^{k-2}) + 2c2^k \\
&= 2^2\left(2T(2^{k-3}) + c2^{k-2}\right) + 2c2^k \\
&= 2^3 T(2^{k-3}) + 3c2^k \\
&\vdots \\
&= 2^k T(1) + kc2^k \\
&= c'n + cn\log_2(n) \\
&= O(n\log(n)).
\end{aligned}
$$

Recursion-tree method

- In a recursion tree, each node represents the cost of a single subproblem somewhere in the set of recursive function invocations.
- You typically sum the costs within each level of the tree to obtain the per-level costs, and then you sum all the per-level costs to determine the total cost of all levels of the recursion.
- A recursion tree is best used to generate intuition for a good guess, which you can then verify by the substitution method.

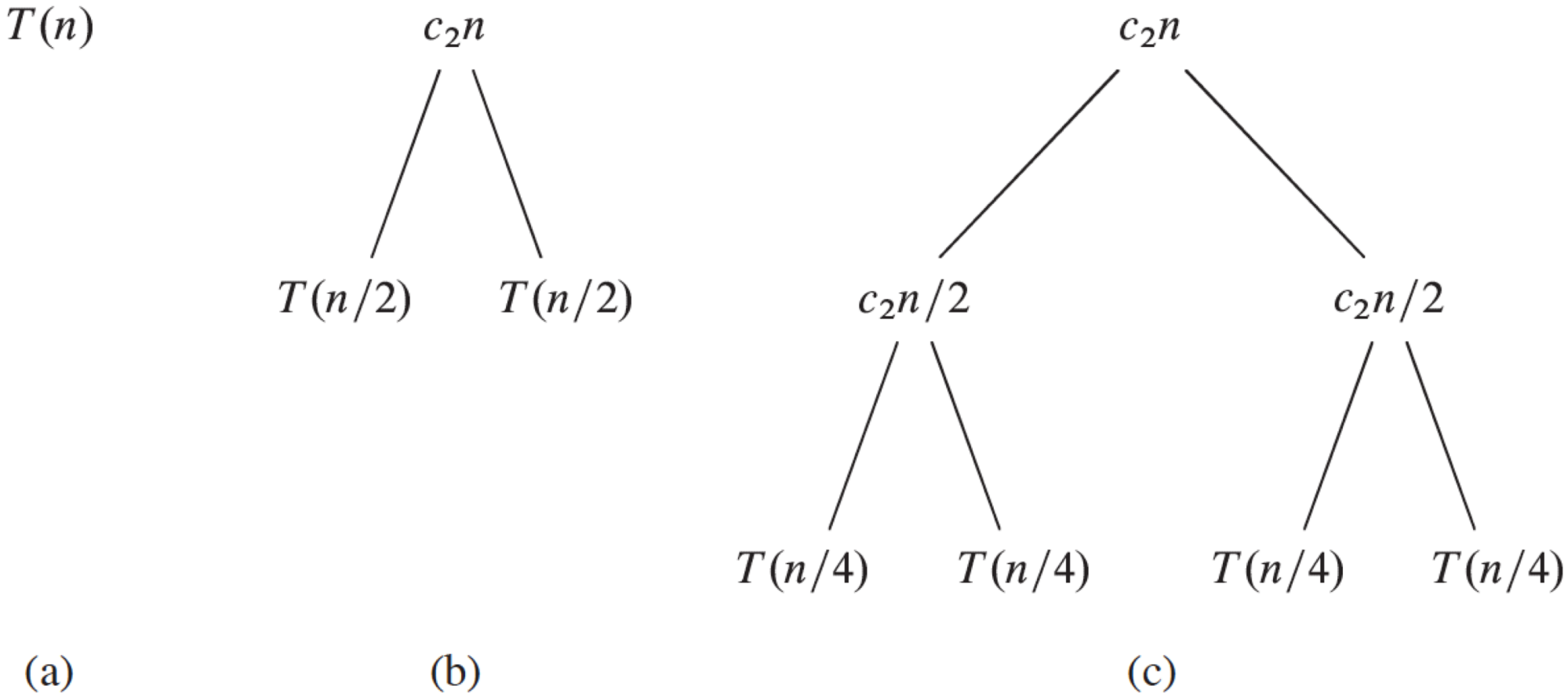
MERGE-SORT running time

$$T(n) = 2T(n/2) + \Theta(n)$$

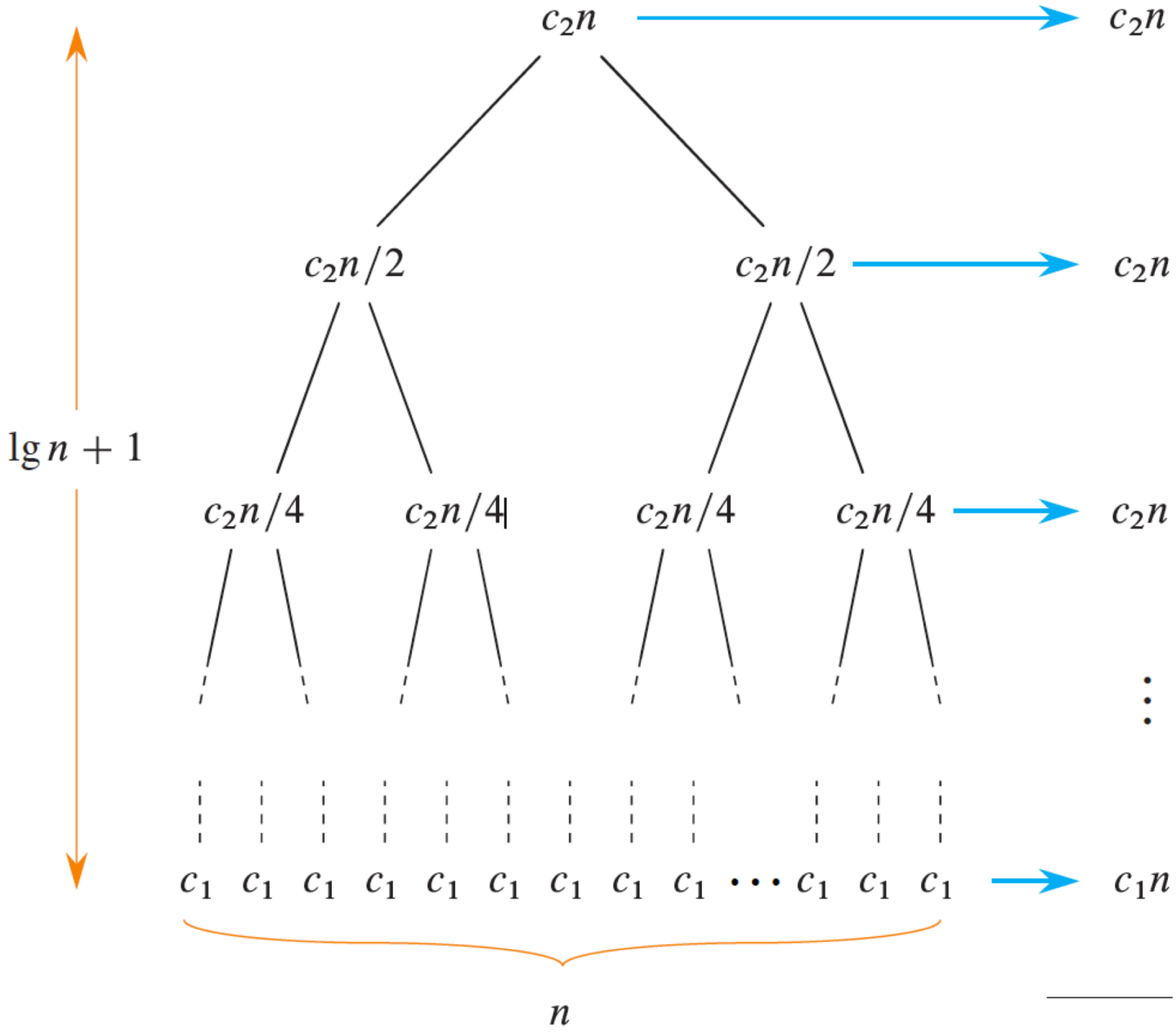
$$T(n) = \begin{cases} c_1 & \text{if } n = 1 , \\ 2T(n/2) + c_2 n & \text{if } n > 1 , \end{cases}$$

MERGE-SORT running time

Recursion tree:



MERGE-SORT running time



MERGE-SORT running time

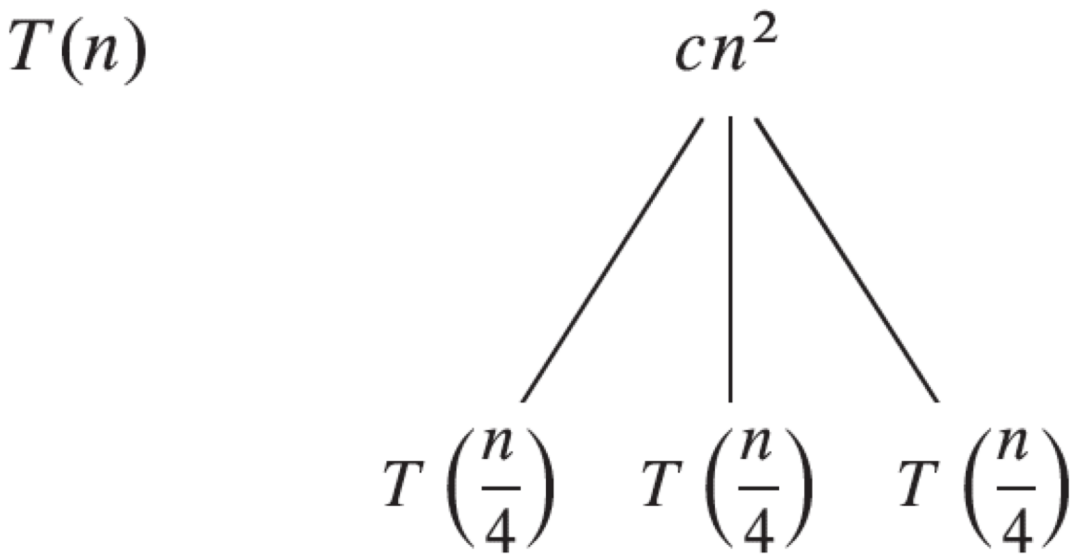
$$c_2 n \lg n + c_1 n = \Theta(n \lg n).$$

$$T(n) = \Theta(n \lg n)$$

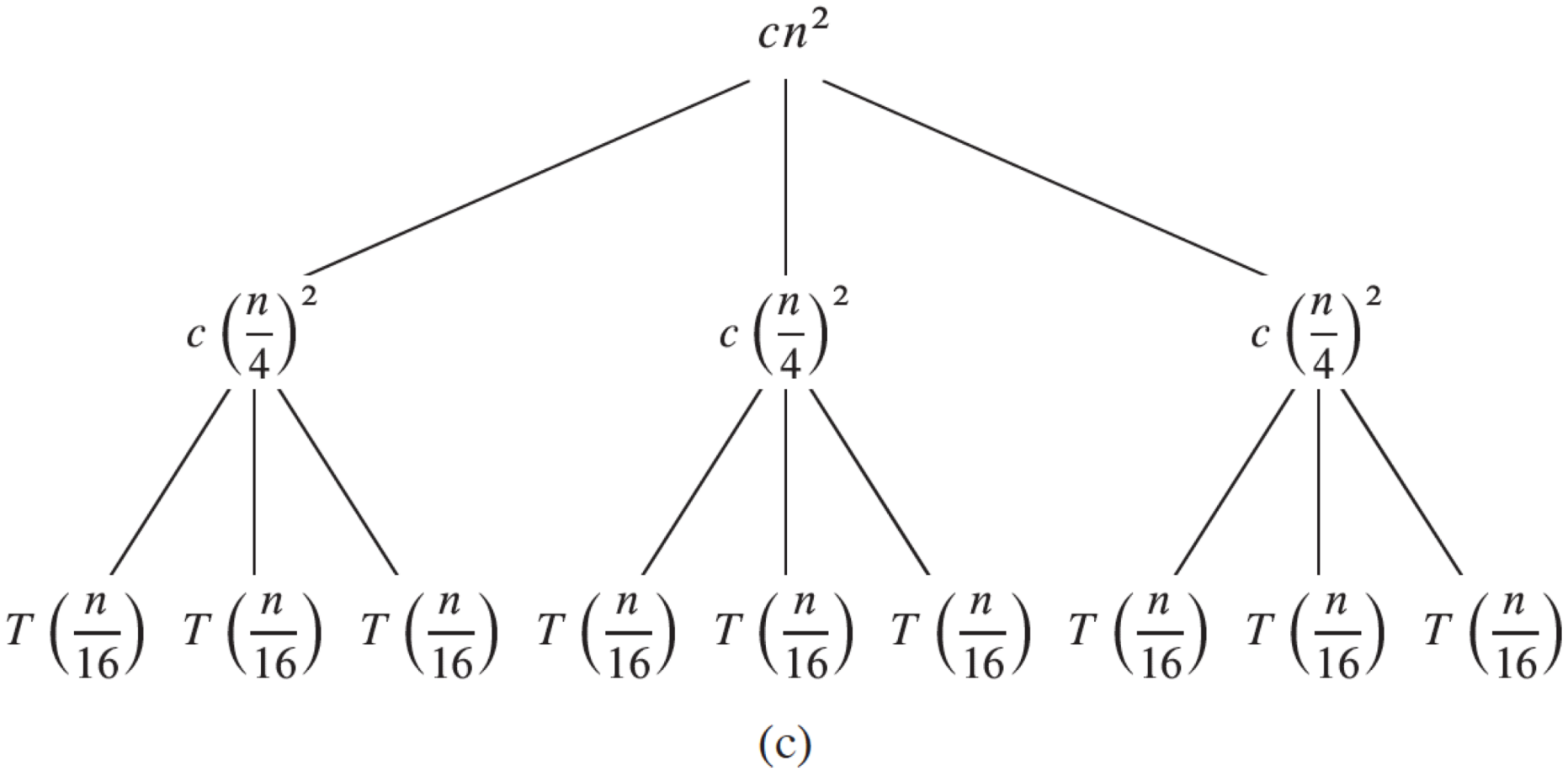
The recursion-tree method, example:

$$T(n) = 3T(n/4) + \Theta(n^2)$$

The recursion-tree method, example:

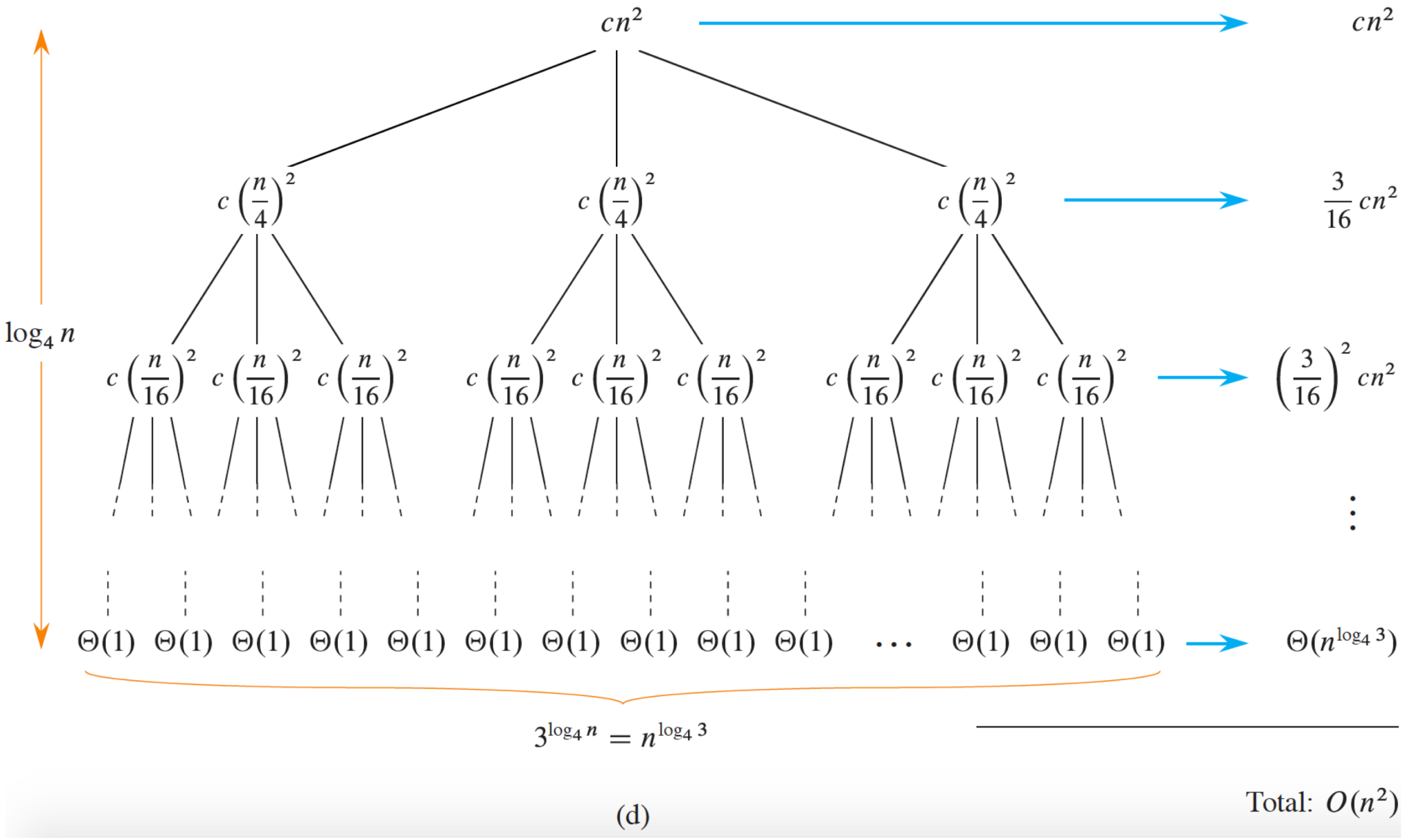


The recursion-tree method, example:



(c)

The recursion-tree method, example:



(d)

The recursion-tree method, example:

What's the height of the recursion tree? The subproblem size for a node at depth i is $n/4^i$. As we descend the tree from the root, the subproblem size hits $n = 1$ when $n/4^i = 1$ or, equivalently, when $i = \log_4 n$. Thus, the tree has internal nodes at depths $0, 1, 2, \ldots, \log_4 n - 1$ and leaves at depth $\log_4 n$.

Part (d) of Figure 4.1 shows the cost at each level of the tree. Each level has three times as many nodes as the level above, and so the number of nodes at depth i is 3^i. Because subproblem sizes reduce by a factor of 4 for each level further from the root, each internal node at depth $i = 0, 1, 2, \ldots, \log_4 n - 1$ has a cost of $c(n/4^i)^2$. Multiplying, we see that the total cost of all nodes at a given depth i is $3^i c(n/4^i)^2 = (3/16)^i cn^2$. The bottom level, at depth $\log_4 n$, contains $3^{\log_4 n} = n^{\log_4 3}$ leaves (using equation (3.21) on page 66). Each leaf contributes $\Theta(1)$, leading to a total leaf cost of $\Theta(n^{\log_4 3})$.

The recursion-tree method, example:

$$
\begin{aligned}
T(n) &= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n} cn^2 + \Theta(n^{\log_4 3}) \\
&= \sum_{i=0}^{\log_4 n} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
&< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
&= \frac{1}{1-(3/16)} cn^2 + \Theta(n^{\log_4 3}) && \text{(by equation (A.7) on page 1142)} \\
&= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\
&= O(n^2) && (\Theta(n^{\log_4 3}) = O(n^{0.8}) = O(n^2)) .
\end{aligned}
$$